

1.2.1 Keywords

The following keywords are reserved and may not be used as an identifier for any other purpose.

<code>auto</code>	<code>double</code>	<code>int</code>	<code>long</code>
<code>break</code>	<code>else</code>	<code>long</code>	<code>switch</code>
<code>case</code>	<code>enum</code>	<code>register</code>	<code>typedef</code>
<code>char</code>	<code>extern</code>	<code>return</code>	<code>union</code>
<code>const</code>	<code>float</code>	<code>short</code>	<code>unsigned</code>
<code>continue</code>	<code>for</code>	<code>signed</code>	<code>void</code>
<code>default</code>	<code>goto</code>	<code>sizeof</code>	<code>volatile</code>
<code>do</code>	<code>if</code>	<code>static</code>	<code>while</code>

1.2.2 Variables

A variable may be defined using any uppercase or lowercase character, a numerical digit (0 through 9), and the underscore character (`_`). The first character of the variable may not be a numerical digit or underscore. Variable names are case sensitive.

The scope of the variable (where it can be used), is determined by where it is defined. If it is defined outside any block or list of parameters, then it has *file scope*. This means it may be accessed anywhere in the current source code file. This is normally called a global variable and is normally defined at the top of the source code. All other types of variables are local variables. If a variable is defined in a block (encapsulated with `{` and `}`), then its scope begins when the variable is defined and ends when it hits the terminating `}`. This is called *block scope*. If the variable is defined in a function prototype, then the variable may only be accessed in that function. This is called *function prototype scope*.

Access to variables outside of their file scope can be made by using *linkage*. Linkage is done by placing the keyword `extern` prior to a variable declaration. This allows a variable that is defined in another source code file to be accessed.

Variables defined within a function scope have *automatic storage duration*. The life of the variable is determined by the life of the function. Space is allocated at the beginning of the function and terminated at the end of the function. *Static storage duration* can be obtained by placing the keyword `static` in front of the variable declaration. This causes the variable's space to be allocated when the program starts up and is kept during the life of the program. The value of the variable is preserved during subsequent calls to the function that defines it. Variables with file scope are automatically static variables.

A variable is defined by the following:

storage-class-specifier type-specifier variable-names,...

The storage-class-specifier can be one of the following:

- `typedef` The symbol name "*variable-name*" becomes a type-specifier of type "*type-specifier*". No variable is actually created, this is merely for convenience.
- `extern` Indicates that the variable is defined outside of the current file. This brings the variables scope into the current scope. No variable is actually created by this.
- `static` Causes a variable that is defined within a function to be preserved in subsequent

calls to the function.

auto Causes a local variable to have a local lifetime (default).

register Requests that the variable be accessed as quickly as possible. This request is not guaranteed. Normally, the variable's value is kept within a CPU register for maximum speed.

The type-specifier can be one of the following:

void	Defines an empty or NULL value whose type is incomplete.
char, signed char	Variable is large enough to store a basic character in the character set. The value is either signed or nonnegative.
unsigned char	Same as char, but unsigned values only.
short, signed short, short int	Defines a short signed integer. May be the same range as a normal int, or half the bits of a normal int.
unsigned short, unsigned short int	Defines an unsigned short integer.
int, signed, signed int, or no type specifier	Defines a signed integer. If no type specifier is given, then this is the default.
unsigned int, unsigned	Same as int, but unsigned values only.
long, signed long, long int, signed long int	Defines a long signed integer. May be twice the bit size as a normal int, or the same as a normal int.
unsigned long, unsigned long int	Same as long, but unsigned values only.
float	A floating-point number. Consists of a sign, a mantissa (number greater than or equal to 1), and an exponent. The mantissa is taken to the power of the exponent then given the sign. The exponent is also signed allowing extremely small fractions. The mantissa gives it a finite precision.
double	A more accurate floating-point number than float. Normally twice as many bits in size.
long double	Increases the size of double.

Here are the maximum and minimum sizes of the type-specifiers on most common implementations. Note: some implementations may be different.

Type	Size	Range
unsigned char	8 bits	0 to 255
char	8 bits	-128 to 127
unsigned int	16 bits	0 to 65,535
short int	16 bits	-32,768 to 32,767
int	16 bits	-32,768 to 32,767
unsigned long	32 bits	0 to 4,294,967,295
long	32 bits	-2,147,483,648 to 2,147,483,647
float		

	32 bits	$1.17549435 * (10^{-38})$ to $3.40282347 * (10^{+38})$
<code>double</code>	64 bits	$2.2250738585072014 * (10^{-308})$ to $1.7976931348623157 * (10^{+308})$
<code>long double</code>	80 bits	$3.4 * (10^{-4932})$ to $1.1 * (10^{4932})$

Examples:

```
int bob=32;
```

Creates variable "bob" and initializes it to the value 32.

```
char loop1,loop2,loop3='\x41';
```

Creates three variables. The value of "loop1" and "loop2" is undefined. The value of loop3 is the letter "A".

```
typedef char boolean;
```

Causes the keyword "boolean" to represent variable-type "char".

```
boolean yes=1;
```

Creates variable "yes" as type "char" and sets its value to 1.

1.2.3 Enumerated Tags

Enumeration allows a series of constant integers to be easily assigned. The format to create a enumeration specifier is:

```
enum identifier {enumerator-list};
```

Identifier is a handle for identification, and is optional.

Enumerator-list is a list of variables to be created. They will be constant integers. Each variable is given the value of the previous variable plus 1. The first variable is given the value of 0.

Examples:

```
enum {joe, mary, bob, fran};
```

Creates 4 variables. The value of joe is 0, mary is 1, bob is 2, and fran is 3.

```
enum test {larry, floyd=20, ted};
```

Creates 3 variables with the identifier test. The value of larry is 0, floyd is 20, and ted is 21.

1.2.4 Arrays

Arrays create single or multidimensional matrices. They are defined by appending an integer encapsulated in brackets at the end of a variable name. Each additional set of brackets defines an additional dimension to the array. When addressing an index in the array, indexing begins at 0 and ends at 1 less than the defined array. If no initial value is given to the array size, then the size is determined by the initializers. When defining a multidimensional array, nested curly braces can be used to specify which dimension of the array to initialize. The outermost nest of curly braces defines the leftmost dimension, and works from left to right.

Examples:

```
int x[5];
```

Defines 5 integers starting at x[0], and ending at x[4]. Their values are undefined.

```
char str[16]="Blueberry";
```

Creates a string. The value at str[8] is the character "y". The value at str[9] is the null character. The values from str[10] to str[15] are undefined.

```
char s[]="abc";
```

Dimensions the array to 4 (just long enough to hold the string plus a null character), and stores the string in the array.

```
int y[3]={4};
```

Sets the value of y[0] to 4 and y[1] and y[2] to 0.

```
int joe[4][5]={
    {1,2,3,4,5},
    {6,7,8,9,10},
    {11,12,13,14,15}
};
```

The first row initializes joe[0], the second row joe[1] and so forth. joe[3] is initialized to 5 zeros.

The same effect is achieved by:

```
int joe[4][5]={1,2,3,4,5,6,7,8,9,10,11,12,13,14,15};
```

1.2.5 Structures and Unions

Structures and unions provide a way to group common variables together. To define a structure use:

```
struct structure-name {
    variables,...
} structure-variables,...;
```

Structure-name is optional and not needed if the structure variables are defined. Inside it can contain any number of variables separated by semicolons. At the end, *structure-variables* defines the actual names of the individual structures. Multiple structures can be defined by separating the variable names with commas. If no *structure-variables* are given, no variables are created. *Structure-variables* can be defined separately by specifying:

```
struct structure-name new-structure-variable;
```

new-structure-variable will be created and has a separate instance of all the variables in *structure-name*.

To access a variable in the structure, you must use a record selector (.).

Unions work in the same way as structures except that all variables are contained in the same location in memory. Enough space is allocated for only the largest variable in the union. All other variables must

share the same memory location. Unions are defined using the union keyword.

Examples:

```
struct my-structure {
    int fred[5];
    char wilma, betty;
    float barny=1;
};
```

This defines the structure my-structure, but nothing has yet been done.

```
struct my-structure account1;
```

This creates account1 and it has all of the variables from my-structure. account1.barny contains the value "1".

```
union my-union {
    char character_num;
    int integer_num;
    long long_num;
    float float_num;
    double double_num;
} number;
```

This defines the union number and allocates just enough space for the variable double_num.

```
number.integer_num=1;
```

Sets the value of integer_num to "1".

```
number.float_num=5;
```

Sets the value of float_num to "5".

```
printf("%i",integer_num);
```

This is undefined since the location of integer_num was overwritten in the previous line by float_num.

1.2.6 Constants

Constants provide a way to define a variable which cannot be modified by any other part in the code. Constants can be defined by placing the keyword `const` in front of any variable declaration. If the keyword `volatile` is placed after `const`, then this allows external routines to modify the variable (such as hardware devices). This also forces the compiler to retrieve the value of the variable each time it is referenced rather than possibly optimizing it in a register.

Constant numbers can be defined in the following way:

Hexadecimal constant:

`0x hexadecimal digits...`

Where *hexadecimal digits* is any digit or any letter **A** through **F** or **a** through **f**.

Decimal constant:

Any number where the first number is not zero.

Octal constant:

Any number where the first number must be zero.

Floating constant:

A fractional number, optionally followed by either `e` or `E` then the exponent.

The number may be suffixed by:

`U` or `u`:

Causes the number to be an unsigned long integer.

`L` or `l`:

If the number is a floating-point number, then it is a long double, otherwise it is an unsigned long integer.

`F` or `f`:

Causes the number to be a floating-point number.

Examples:

```
const float PI=3.141;
```

Causes the variable PI to be created with value 3.141. Any subsequent attempts to write to PI are not allowed.

```
const int joe=0xFFFF;
```

Causes joe to be created with the value of 65535 decimal.

```
const float penny=7.4e5;
```

Causes penny to be created with the value of 740000.000000.

1.2.7 Strings

Strings are simply an array of characters encapsulated in double quotes. At the end of the string a null character is appended.

Examples:

`"\x41"` and `"A"` are the same string.

```
char fred[25]="He said, \"Go away!\";
```

The value at fred[9] is a double quote. The value at fred[20] is the null character.

1.2.8 sizeof Keyword

Declaration:

```
size_t sizeof expression
```

or

```
size_t sizeof (type)
```

The sizeof keyword returns the number of bytes of the given expression or type. `size_t` is an unsigned integer result.

Example:

```
printf("The number of bytes in an int is %d.\n",sizeof(int));
```